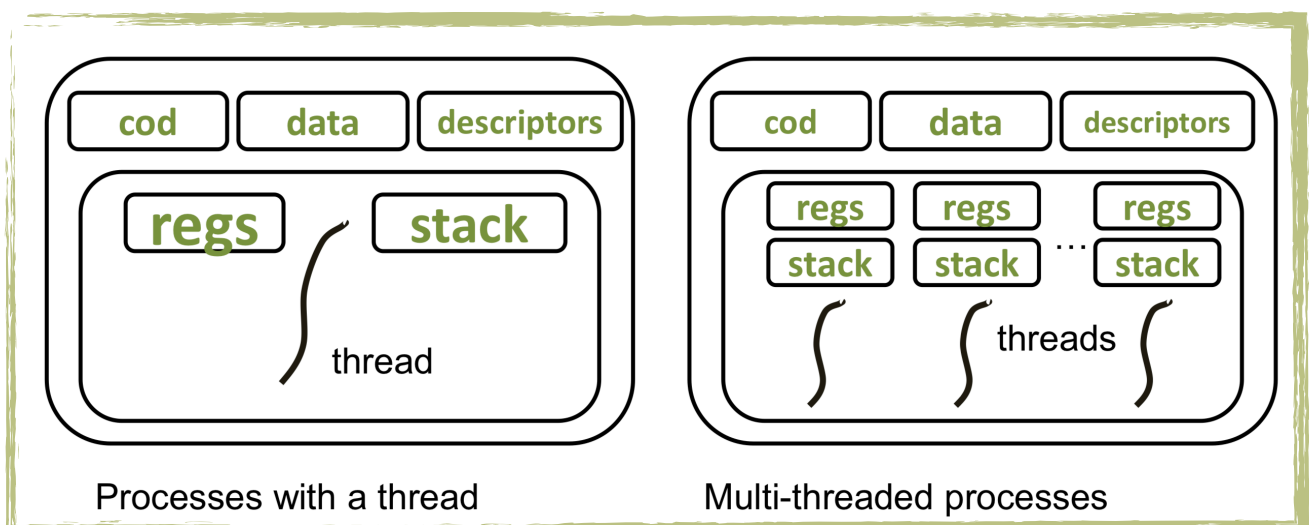


Laboratorul 12

Modelul Client/Server TCP/IP Concurrent Multithreaded. Exemple de implementare.

1. Fire de executie

- **Threads** (firele de executie) pot fi vazute ca cea mai mica sectiune de cod ce poate fi administrata de sistemul de operare;
- Daca in cazul proceselor copil se creea un nou proces propriu-zis, threadurilor nu li se aloca memorie, pid etc. Mai sunt numite si “lightweight processes”;
- Threadurile si procesul principal impart aceeasi memorie, ceea ce implica ca variabilele de la nivelul procesului sunt accesibile oricarui thread;
- Un proces contine unul sau mai multe threaduri;
- Deoarece threadurile impart memoria si au access la codul executabil al procesului, de cele mai multe ori apare fenomenul “race condition”;
- **Race condition** este declansat atunci cand unul sau mai multe threaduri incearca sa acceseze si sa modifice in acelasi timp resurse shared;



- Pentru a administra accesul la resurse si a evita aparitia de “race condition” se pot folosi mecanisme de tip mutex, semafoare etc.

2. Implementarea firelor de executie si primitive de baza

- Standardul **Pthreads (Posix Threads)** defineste un API pentru a crea si a manipula fire de executie;

```
#include <pthread.h>
```

```
//crearea unui fir de executie nou
```

```
int pthread_create( pthread_t *tid, const pthread_attr_t *attr, void *(*func) (void *),  
void *arg);
```

Parametri:

- **tid** - identificatorul threadului (de cele mai multe ori un unsigned int);
- **attr** - specifica atributele threadului nou creat (dimensiunea stivei, prioritatea); valoarea default este NULL;
- **(*func) (void *)** - referinta catre functia ce va fi executata de thread;
- **arg** - argument ce este pasat functiei care va fi executata de catre thread;

In caz de **succes**, **pthread_create()** returneaza 0.

In caz de **eroare**, **pthread_create()** returneaza un numar de eroare si continutul lui **tid** este nedefinit.

```
#include <pthread.h>
```

```
// utilizata pentru a astepta un thread sa isi termine executia
```

```
int pthread_join( pthread_t *tid, void **status );
```

Parametri:

- **tid** - identificatorul threadului (de cele mai multe ori un unsigned int);
- **status** - valoarea returnata de functia executata de thread;

In caz de **succes**, pthread_join() returneaza 0.

In caz de **eroare**, pthread_join() returneaza un numar de eroare.

```
#include <pthread.h>
```

```
// utilizata pentru a elibera toate resursele la finalul executiei threadului
```

```
int pthread_detach( pthread_t *tid);
```

Parametri:

- **tid** - identificatorul threadului (de cele mai multe ori un unsigned int);

In caz de **succes**, pthread_detach() returneaza 0.

In caz de **eroare**, pthread_detach() returneaza un numar de eroare.

```
#include <pthread.h>
```

```
// utilizata pentru a returna ID-ul threadului apelant
```

```
pthread_t pthread_self( pthread_t *tid);
```

```
#include <pthread.h>
```

```
//incheierea executiei unui thread
```

```
int pthread_exit( void *status);
```

Parametri:

- **status** - valoarea de exit a thread-ului;

In caz de **succes**, `pthread_exit()` returneaza 0.

In caz de **eroare**, `pthread_exit()` returneaza un numar de eroare.

- Thread-urile pot fi de doua tipuri:
 - **Joinable** - cand thread-ul isi termina executia codul de return si ID-ul acestuia sunt tinute in memorie pana cand `pthread_join` este apelat;
 - **Detached** - cand thread-ul isi termina executia toate resursele sunt dealocate, fara a mai fi nevoie sa apelam `pthread_join`;
- Threadurile isi pot **incheia executia** din doua motive:
 - Termina de executat functia de care este responsabil;
 - Se termina procesul principal (functia `main()` returneaza) sau unul dintre threaduri a apelat `exit()`;

3. Implementarea unui model client/server TCP/IP concurent

Server TCP/IP care creeaza un nou fir de executie pentru fiecare client conectat:

<https://profs.info.uaic.ro/~computernetworks/files/NetEx/S12/ServerConcThread/servTcpConcTh2.c>

Server TCP/IP prethreaded care foloseste mutexuri pentru a proteja accept():

<https://profs.info.uaic.ro/~computernetworks/files/NetEx/S12/ServerPreThread/servTcpPreTh.c>

Client TCP/IP va fi folosit pentru a comunica cu ambele servere:

<https://profs.info.uaic.ro/~computernetworks/files/NetEx/S12/ServerPreThread/cliTcpNr.c>

Pentru a **compila** sursele C, veti folosi urmatoarele comenzi:

```
gcc servTcpConcTh2.c -lpthread -o servTcpConcTh2
```

```
gcc servTcpPreTh.c -lpthread -o servTcpPreTh
```

```
gcc cliTcpNr.c -o cliTcpNr
```